

8.2小组模拟面

1.静态库和动态库加载机制

首先程序员写的.c文件，经过编译期和汇编期变成.o文件，程序中用到的静态库函数，是在**链接期引入的**

- 如果是静态库（.a（Linux, Unix, macOS）.lib（windows）），编译期不做额外操作，链接时链接器会把.o文件和静态库文件一起作为输入，分析main.o中未解析符号，并在静态库中查找匹配的，如果找到了就将代码和数据提取并合并到exe文件中（**未使用的不合并**）
- 如果是动态库（.so（Linux, Unix, macOS）.dll（windows）），编译期会记录符号依赖关系（函数名，对应的动态库），生成符号表，运行时遇到无法解析的符号根据符号表查对应函数

引入库的报错问题：

- 如果程序只有静态库，如果函数不识别，即在静态库中也没有，就是编译错误。
- 如果只引入了动态库，运行时才判断出没有，符号表也没有，此时是运行时错误
- 如果既有静态库又有动态库，遇到函数没有先找静态库，编译时不报错，然后找动态库，都没有就报运行错误

2.std::move底层如何实现的

用static_cast进行类型转换，把左值转换为右值引用

```
1  template <typename T>
2  typename remove_reference<T>::type&& move(T&& t)
3  {
4      return static_cast<typename remove_reference<T>::type&&>(t);
5  }
```

其中remove_reference用于去掉引用，获得其对应的右值类型，不管传入的是左值引用还是右值引用，它都只会返回这个值去掉引用后的类型。我们以int为例，不管传入int&还是int&&，经过remove_reference后，返回的都是int

3.介绍std::move失效的情景

失效的场景：

- 接收这个空间的对象禁用了移动构造，移动赋值函数，就会失效
- 对非左值使用了move
- lambda表达式捕获，期望操作的对象和捕获的不是同一个

4.构造函数能否声明为虚函数

构造函数：

构造函数不能定义为虚函数。虚函数存储在虚函数表中，通过虚函数指针去指向虚函数表来调用。

而虚指针存在对象的前四个字节，如果构造函数是虚函数，这时对象还没被初始化，还没有虚指针，就无法找到虚函数表，就无法调用虚函数，所以构造函数不能是虚函数。

构造函数必须明确知道要创建的对象类型，因为它负责初始化对象的内存布局和成员变量。如果构

造函数是虚函数，虚函数是运行时确定类型的，这样矛盾。

5.什么函数可以声明为虚函数，什么函数不可以

可以：

- **普通成员函数：**声明为虚函数，允许子类重写
- **析构函数：**确保通过父类指针删除子类对象时，正确调用子类析构函数

不可以：

- **构造函数：**构造函数用于初始化对象，此时对象的动态类型尚未完全确定（子类部分未初始化），

无法实现动态绑定。

- **静态成员函数：**静态成员函数属于类而非对象，没有 this 指针，无法通过对象动态调用，与虚函

数机制矛盾。

- **友元函数**：友元函数不是类的成员，无法被继承或重写，因此不能声明为虚函数。
- **内联函数**：内联函数运行在编译期，而虚函数是运行时确定类型
- **模板函数**：模板函数的类型在编译时确定，而虚函数依赖运行时多态

6.万能引用的底层，如何实现完美转发

```
template void func(T&& tmpvalue) {...}
```

万能引用本质是个模版，参数是&&，既可以传左值又能传右值（保留原始值类别：比如我传 &，此时在&&作用下还是&，右值引用一样是&&，左值或右值直接变成&&，都延长了生命周期，且没有发生额外的拷贝），万能引用经常和forward结合，实现完美转发

为什么要完美转发：避免不必要的拷贝或移动，提升性能（尤其对于大型对象或资源句柄），且支持把参数原封不动传给其他函数

7.push_back与emplace_back，对push_back底层机制的纠正

```
void push_back(const T& x); // 拷贝构造
```

```
void push_back(T&& x); // 移动构造
```

```

1  template<typename T>
2  void vector<T>::push_back(const T& value) {
3      if (size_ >= capacity_) {          // 当前空间不够了
4          reserve(capacity_ == 0 ? 1 : capacity_ * 2); // 扩容策略：通常是翻倍或至少+1
5      }
6      new (data_ + size_) T(value);      // 在尾部构造一个新元素（调用拷贝构造函数）
7      ++size_;                          // 元素个数 +1
8  }
9
10 template<typename T>
11 void vector<T>::push_back(T&& value) {
12     if (size_ >= capacity_) {
13         reserve(capacity_ == 0 ? 1 : capacity_ * 2);
14     }
15     new (data_ + size_) T(std::move(value)); // 移动构造
16     ++size_;
17 }
```

push_back不是一定会多一次构造，具体要看是否传入右值引用，如果是则采用移动构造，此时开销和emplace_back一致

```
1 MyClass obj(1, "hello");
2 vec.push_back(std::move(obj));    // 移动构造
3 vec.emplace_back(std::move(obj)); // 也是移动构造（但通常你不会这么用 emplace_back）
4 //但是对于这种右值引用move转移使用效率一样
```

除此之外emplace_back支持可变参数模板，也就是可以直接通过传入类的类型和类构造函数的参数，在类的末尾空间动态创建类，但是push_back不支持直接构造类，所以对于装类的对象的vector，一定会多一次拷贝

```
1 vec.push_back(MyClass(1, "tmp")); // 构造临时对象，然后拷贝/移动进去（可能发生拷贝构造）
2 vec.emplace_back(1, "tmp");      // 直接在 vector 中构造，无临时对象
3 //对于上面的情况，emplace_back效率更高
```

8.union是可继承的么？

union默认访问修饰符是public，不存在继承关系，类函数不占union空间，但属于union（和类一样）

9.enum和enum class的区别

- enum是普通枚举，如果不显示指定，则第一个枚举量值为0，后面递增，如果声明多个enum，比如enum e1{red},e2{blue};则red和blue都是0，**为了更明显区分出是哪个enum的，可以用enum class（使用时必须加作用域，比如e1::red）**
- enum class可以显示指定枚举类占的空间大小，比如enum class SmallColor : uint8_t { //使用 8-bit 存储 Red, Green, Blue };而enum默认为int

10.HTTP和RPC的区别

两者都是基于TCP的，早期TCP由于是数据流，所以需要定义消息格式确定消息边界，因而诞生了RPC和HTTP

RPC是早期公司内部自己研发的访问自家服务器的，本质是一种调用方式，用于远程调用自家服务器向外提供的接口，它相较于HTTP定制化更高，且不存在包头冗余的字段，访问起来速率快

但HTTP允许互联网服务器之间跨平台访问，且随着HTTP的迭代，优势逐渐明显

11.索引下推

这是一个和索引覆盖及其相似的概念，**索引覆盖指的是查询的数据恰好就在一棵索引树中，此时就不用回表了**

假如有个联合索引 (a, b, c)

select a, b from table where a=1; 此时走联合索引，这个联合索引树里恰好有 (a, b) 所以直接返回结果了，这就是索引覆盖

索引下推指的是过滤条件所需要的字段恰好在索引树中能得到判断，而不需要回表再判断一次

同样的联合索引 (a, b, c)

select a from table where a>1 and b=2;

对于这个sql语句，我们在联合索引中找到所有a>1的项，就能立马在索引树中判断出b的值，从而得到验证，而不用回表以a的值作为主键再查一次对应b为多少，这就是索引下推

索引覆盖强调查询的数据在联合索引中，索引下推强调查询条件在联合索引中